

KAIST GSEB · SBML LAB · INTERN ONBOARDING

AI-Assisted Bioinformatics Training

6-Week Claude Code Curriculum for New Lab Members

SBML Lab

Prepared for Prof. Donghyuk Kim

v1.0 · 2026.05 · KAIST GSEB

| Contents

01	Overview	03
02	Module Descriptions	04
03	Mini-Project	07
04	Technical Environment	09

Overview

A 6-week curriculum teaching new SBML Lab interns to use [Claude Code](#) within real bioinformatics workflows — from GFF parsing and read alignment through motif discovery — delivered entirely on GitHub Codespaces with no local installation required.

Schedule

Week	Module	Topic
1	Module 1	Introduction to Claude Code
2	Module 2	File Parsing and Data Processing
3	Module 3	NGS Alignment Pipeline
4	Module 4	Literature Integration, Biopython, and Motif Analysis
5–6	Module 5	Independent Mini-Project

Lab-Provided Skills

Skill	When to use
<code>/log</code>	End of every session — records progress and learnings across all six weeks.
<code>/debug</code>	When any command or script fails — structured reproduce → fix → verify protocol. Required before asking for help.
<code>/explain</code>	Before external search — explains tools, algorithms, and concepts in lab context.

All code cells start empty. No answers are provided in advance — interns use Claude Code to figure out what to run and navigate public databases themselves.

| Module Descriptions

WEEK 1 Module 1 — Introduction to Claude Code

Objectives: Understand Claude Code's operational model (terminal, files, commands — not a chatbot); write effective prompts; develop the habit of verifying Claude's output critically; use the three lab skills; make a first commit.

1. Write a script from natural language. Describe a star-triangle pattern; ask Claude Code to produce and refine the script.
2. Debug a broken script. Use `/debug` on a countdown script with an intentional `range()` bug — no exception, wrong output.
3. Read and invoke the three skills. Read each skill file, invoke it, and answer when and why to use it.
4. Extend a skill. Add one detail from `lab-context.md` to `/explain` — skills are plain Markdown files.

Deliverable: a `/log` session entry.

WEEK 2 Module 2 — File Parsing and Data Processing

Objectives: Generate and refine GFF parsing scripts from natural language; diagnose pandas loading and indexing errors.

All exercises use the lab's *E. coli* K-12 MG1655 annotation GFF (`ec_annotation_20100903_DHK_cSRNA_with_ortho.gff`) — the same file used in Modules 4 and 5.

1. Natural language → GFF parsing script. Describe a filtering task; at least two refinement prompts required.
2. Debug a pandas script. Two bugs: missing `header=None` and string column access instead of integer index. Use `/debug` to reproduce and fix both.
3. When to use `iterrows()`. Write a one-sentence rule first, then verify with Claude Code.

WEEK 3 Module 3 — NGS Alignment Pipeline

Objectives: Understand bowtie2 and samtools at a conceptual level; use plan mode to generate and review a full pipeline; write `makegff.py` using pysam.

Covers the complete single-end ChIP-exo workflow: reference indexing → bowtie2 alignment → samtools sort/index → GFF conversion for MetaScope. All concepts are explained through Claude Code prompts, not pre-written text.

1. Understand bowtie2. Ask Claude Code how it finds alignments; write a 3-sentence summary before running anything.
2. Sort-before-index reasoning. Write the explanation yourself first, then verify.
3. Plan mode pipeline. Generate the full pipeline with Shift+Tab; review every step and identify at least one change before approving.
4. Download ChIP-exo reads. Navigate GEO series GSE54901, find the SRR accession, and download — no commands given.
5. Decode a SAM FLAG. Choose any FLAG value; decode its bitwise components with Claude Code.
6. Write `makegff.py`. Use pysam to output one GFF line per aligned read:

```
NC_000913.3 makegff read <start> <end> 1 <strand> . name=
<read_name>
```

WEEK 4 Module 4 — Literature, Biopython, and Motif Analysis

Objectives: Extract structured information from a published paper; retrieve GEO data; write a Biopython sequence extraction script; run MEME; interpret motif results biologically.

Built around Seo et al. 2014 (GSE54901) — FUR ChIP-exo in *E. coli*. The expected Fur box motif is known from literature, making it a verifiable training target.

1. Read Seo et al. 2014. Write a 3-sentence summary (experiment, condition, finding) before the concept cell states anything.
2. Cross-check the FUR regulon. Identify other published reference gene sets and how they were determined.
3. Understand MEME. Write how it works and what E-value means; verify with Claude Code.
4. Download data from GEO. Provide the series URL; ask Claude Code to retrieve the peak GFF and reference FASTA.

5. Biopython sequence extraction. Extract ± 50 bp windows around each peak summit; write to multi-FASTA for MEME input.
6. Run MEME and interpret. Build the command with Claude Code; assess whether the top motif is biologically meaningful.
7. TSS distance analysis. Compute distance from each FUR site to the nearest annotated TSS using the lab GFF.

Mini-Project

Weeks 5–6: an original analysis on the same FUR ChIP-exo data from Module 4. No new dataset. Interns choose their own biological question, build the pipeline, and write a reproducible report.

Six-Day Milestones

Day	Milestone
Day 1	Literature review complete; biological question drafted
Day 2	Question approved at instructor check-in
Day 3	Analysis plan written; plan-mode pipeline generated and annotated
Day 4	Pipeline running; at least one result figure produced
Day 5	Analysis complete; all figures finalized
Day 6	Report written; notebook submitted

The biological question must be specific and answerable from the data (e.g., "Are FUR binding sites enriched near sRNA genes?"). Vague questions are not accepted. All errors go through `/debug` ; all sessions through `/log` . The report covers question, methods, results, and interpretation in Markdown cells.

Design Principles

1. Interns discover answers through Claude Code, not pre-filled cells. All code cells start empty — including data retrieval commands.
2. Every module uses real lab data and real tools. The same annotation GFF, SRA reads, and GEO datasets that appear in actual lab research.
3. Claude Code is a tool to understand, not a shortcut to avoid understanding. Every module has a conceptual check written before consulting Claude Code; plan mode is required for all pipelines.
4. Critical verification is a habit, not a step. Claude can be wrong. Interns are expected to notice when a result or claim doesn't feel right and investigate.

Technical Environment

GitHub Codespaces · conda env sbml · Python 3.11 · no local setup required.

Tool	Purpose
bowtie2	Short-read alignment
samtools	BAM processing and coordinate sorting
bedtools	Genomic interval operations
sra-tools	SRA / GEO data download
MEME Suite	Motif discovery
Biopython	FASTA parsing and sequence manipulation
pysam	BAM file parsing from Python
pandas · matplotlib · scipy	Data processing and visualization
entrez-direct	NCBI data retrieval via command line
Claude Code CLI	AI coding assistant (requires Claude Pro account)

Pre-Distribution Checklist

1. Copy `ec_annotation_20100903_DHK_cSRNA_with_ortho.gff` into `data/reference/`.
2. Verify GSE54901 peak file is downloadable from GEO.
3. Test-launch a Codespace; confirm all tools including pysam.
4. Confirm interns have Claude Pro accounts before Week 3 (plan mode requires Pro).